Neural Networks as Approximate Smoothers

Degree Dissertation
for the
Master Examination in Data Science in Business and Economics
at the
Faculty of Economics and Social Sciences
of the
Eberhard Karls Universität
Tübingen

Examiner:
Jun.-Prof. Dr. Michael Knaus

Submitted by:
Aaron Sennhenn
Born in Geislingen an der Steige

Date of submission: 30/07/2026

Contents

List of Figures

List of Tables

1 Introduction

Neural networks provide a flexible framework for learning complex nonlinear relationships and have become an important component of modern machine learning. Their predictive flexibility makes them suitable nuisance learners in Double Machine Learning (DML), where machine-learning methods are combined with orthogonal estimation procedures to estimate treatment and structural parameters (Chernozhukov et al., 2018). While DML permits the use of neural networks, their predictions are naturally not expressed in terms of the observed outcomes. This caveat limits their compatibility with the Outcome-Weights framework of Knaus (2024), which requires nuisance predictions to admit a smoother representation in the form of $\hat{y} = Sy$. Motivated by this limitation, this thesis develops two approaches for approximately representing neural network predictions in smoother form. The first, building on Coulombe et al. (2024), uses features learned by a neural network as regressors in an auxiliary ridge regression. Re-estimating the final readout yields an exact smoother representation for the resulting post-processed estimator, although its predictions need not match those of the original network. The second approach applies the telescoping framework of Jeffares et al. (2024), which approximates the network's training trajectory through a sequence of local linear updates. The empirical analysis indicates that both approaches produce useful smoother representations in the evaluated settings. Neural-feature ridge slightly improves predictive performance on average, whereas the telescoping approximation closely reproduces the predictions of the original network. However, constructing the telescoping smoother is computationally expensive and limits its practical applicability considerably (Jeffares et al., 2024). Finally, an application demonstrates how the neural-feature ridge smoother can be applied in the Outcome-Weights framework of Knaus (2024), thereby making outcome-weight diagnostics available for a neural-network-based nuisance estimator. The thesis is organized as follows: Chapter 2 introduces the theoretical background, Chapter 3 develops and discusses the two smoother constructions, Chapter 4 evaluates them empirically, and Chapter 5 applies the neural-feature ridge smoother to Outcome-Weights diagnostics. Chapter 6 concludes this work.

2 Background

This chapter provides the foundation for the methodological approaches developed in this work. Firstly, it introduces neural networks as flexible prediction functions, focusing on their structure, training dynamics, and regularization. It then defines smoother representations as the link between neural network predictions and weighted combinations of observed outcomes.

2.1 Neural networks

Following the general presentation of neural networks in Prince (2023, p. 26–28) and Coulombe et al. (2024), this work considers feed-forward neural networks in the form of multilayer perceptrons (MLPs). Going forward, the term MLP is used for fully connected feed-forward networks, where information moves sequentially from the input layer through multiple hidden layers to a final output layer.

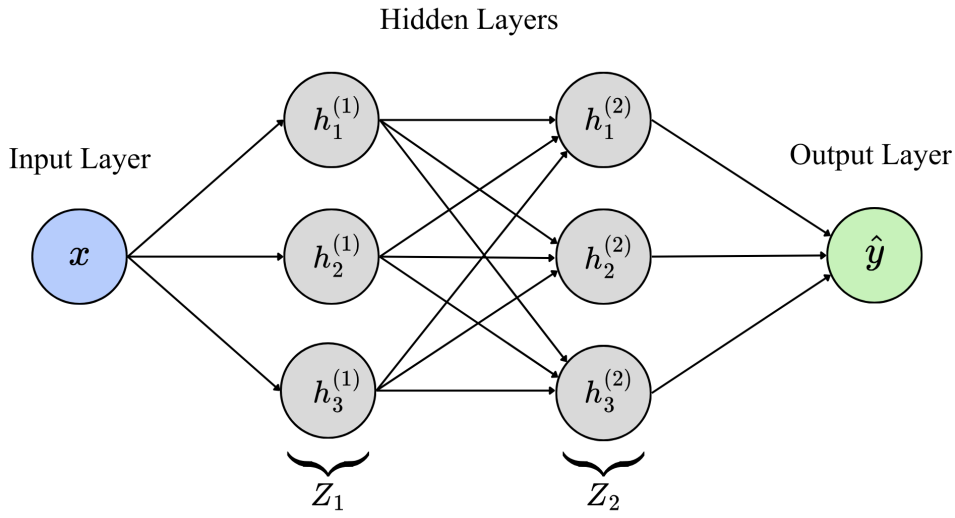

Figure 2.1: Sketch of a feed-forward neural network with two hidden layers.

An MLP takes an input $x \in \mathbb{R}^p$ and maps it to an output prediction $\hat{y}$ through a sequence of layer-wise transformations. Let $Z_{l,i}$ denote the output vector of observation x_i in hidden layer l, where $l = 1, \ldots, L-1$. Each layer consists of several hidden units, also called neurons, whose activations are denoted by

$$Z_{l,i} = \left(h_{1,i}^{(l)}, \ldots, h_{p_l,i}^{(l)} \right), \tag{2.1}$$

where p_l is the number of neurons in layer l. The number of neurons in a hidden layer is known as the layer's width, while the depth of a neural network describes the total number of hidden layers. Starting from the input x_i, the first hidden layer is obtained by forming weighted sums of the input variables, adding a bias term and applying a nonlinear activation function $a(\cdot)$. The choice of activation function determines how the weighted input to a neuron is transformed. Common examples include the sigmoid function or the rectified linear unit (ReLU). For example, the activation of the first neuron in the first hidden layer is computed as

$$h_{1,i}^{(1)} = a\left(b_1^{(1)} + x_i^\top w_1^{(1)} \right), \tag{2.2}$$

where $b_1^{(1)}$ is the bias of neuron 1 in the first hidden layer and $w_1^{(1)} \in \mathbb{R}^p$ is its vector of input weights. The bias and weights are stored in the parameter vector $\theta_l = (b^{(l)}, w^{(l)})$. Collecting all activations in the first hidden layer gives

$$Z_{1,i} = a\Big(\begin{bmatrix} 1 & x_i \end{bmatrix} \theta_1 \Big), \tag{2.3}$$

where the activation function is applied elementwise. Subsequently, the activations of each hidden layer are derived from those of the preceding layer and can be expressed in general form as

$$Z_{l,i} = a\Big(\begin{bmatrix} 1 & Z_{l-1,i} \end{bmatrix} \theta_l \Big). \tag{2.4}$$

Architecture and capacity.

Since the output layer that forms the final prediction does not apply an activation function in a regression setting, the prediction is obtained as a linear readout of the activations from the last hidden layer, also called the penultimate layer:

$$\hat{y} = \begin{bmatrix} 1 & Z_{L-1} \end{bmatrix} \hat{\theta}_L. \tag{2.5}$$

As the network maps inputs to predictions through parameterized transformations, an MLP can also be viewed as a family of functions f_θ. This function perspective is useful for understanding neural networks theoretically. As shown by Prince (2023, p. 29), increasing the number of hidden units can be interpreted as increasing the number of linear regions over which the network can represent different functional behavior.

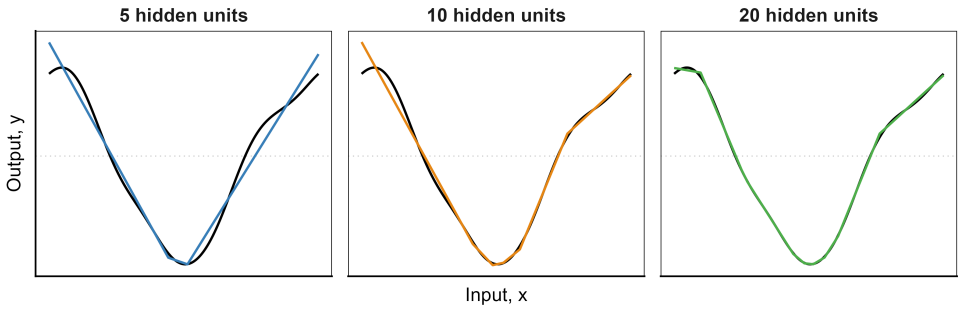

Figure 2.2: Fitting of a one-dimensional function using neural networks with increasing numbers of hidden units. Adapted from Prince (2023, p. 30).

From an approximation-theoretic perspective, neural networks are motivated by this ability to flexibly represent nonlinear functions. The foundational universal approximation theorem proposed by Cybenko (1989) shows that a shallow neural network with sufficient hidden units may approximate a broad class of continuous functions to an arbitrary accuracy. As Suh and Cheng (2025) explain, later approximation-theoretic results further show that, for certain function classes, the approximation quality is not affected by input dimensionality, which is consistent with observations of neural networks' effectiveness in handling high-dimensional data.

However, this is an existence result and does not by itself imply efficient training or good generalization (Suh and Cheng, 2025). This leads to the question of capacity. Increasing the size of the network enlarges the set of functions that can be represented and may therefore reduce approximation error. At the same time, classical statistical learning theory associates high capacity with a greater risk of overfitting (Goodfellow et al., 2016, p. 109). Hence, the performance of neural networks depends not only on approximation capacity, but also on training and regularization.

Training and optimization.

Following Goodfellow et al. (2016, p. 164), training determines the parameter values θ of the fitted neural network. Thus, from a statistical learning perspective, training consists of an estimation process where parameter values $\hat{\theta}$ are selected that minimize an objective over the observed training sample

$$\hat{\theta} = \arg\min_{\theta} \frac{1}{n} \sum_{i=1}^{n} \ell(y_i, f_\theta(x_i)), \tag{2.6}$$

where $f_\theta(x_i)$ is the predictor for observation x_i and $\ell(\cdot, \cdot)$ denotes the observation-level loss function. For regression tasks with squared-error loss, $\ell(f_\theta(x_i), y_i) = (f_\theta(x_i) - y_i)^2$, the objective corresponds to the mean squared error (MSE).

Because neural network objectives generally cannot be minimized in closed form, their parameters are determined using iterative numerical optimization to find $\hat{\theta}$ (Goodfellow et al., 2016, pp. 147–148). A central algorithm for this purpose is stochastic gradient descent (SGD), which extends gradient descent (GD) by estimating the objective's gradient from a randomly selected subset of observations called a mini-batch, rather than the complete training sample. SGD has numerous variants, including momentum, RMSProp, and Adam, but all retain the same underlying principle: parameters are updated iteratively using gradients estimated from mini-batches. These methods differ primarily in how they accumulate, rescale, or adapt the gradients and learning rates used in the update (Goodfellow et al., 2016, pp. 288–302).

At training iteration t, a mini-batch B_t is drawn from the shuffled training data and propagated through the network to obtain predictions $f_{\theta_t}(x_i)$ which are used to compute the mini-batch loss $\mathcal{L}_{B_t}(\theta_t)$

$$\mathcal{L}_{B_t}(\theta_t) = \frac{1}{|B_t|} \sum_{i \in B_t} \ell\left(f_{\theta_t}(x_i), y_i\right). \tag{2.7}$$

The propagation of the mini-batch through the network and the subsequent computation of the loss constitute the forward pass. Before the parameters can be updated, backpropagation recursively applies the chain rule from the output layer towards the input layer to compute the gradient $\nabla_\theta \mathcal{L}_{B_t}(\theta_t)$.

$$\theta_{t+1} = \theta_t - \gamma \nabla_\theta \mathcal{L}_{B_t}(\theta_t) \tag{2.8}$$

Here, $\gamma > 0$ denotes the learning rate which controls the relative size of the parameter update. This computation is called the backward pass: it computes the gradients, while the subsequent optimization step $\theta_t \to \theta_{t+1}$ uses them to update the parameters. Mini-batch updates are performed successively throughout the training sample. When all shuffled observations have been processed once, one epoch is completed. Training generally proceeds over multiple epochs. At selected intervals, the model may be evaluated on a validation set separate from the training set, which is not used for gradient updates but is used to monitor generalization performance and to determine when training should terminate (Goodfellow et al., 2016, pp. 117–118). Training concludes when the maximum number of epochs is reached or when validation performance ceases to improve further. The latter technique is referred to as early stopping and can also act as a form of regularization.

Regularization and generalization.

Goodfellow et al. (2016, p. 116) define regularization as any modification made to a learning algorithm that is aimed at reducing the generalization error but not the training error. Given the limited scope of this thesis, this section highlights only a few of the many regularization methods a neural network model has at its disposal, in particular ones that are essential to understand or help provide further intuition for concepts described in the methodological chapter. Training a neural network reduces the prediction error on the observed sample; however, statistical learning is ultimately concerned with prediction on out-of-sample observations. A highly complex model may achieve a very small training loss by adapting to sample-specific variation that does not generalize. Statistically, regularization can therefore be understood as controlling the effective complexity of the neural network and trading additional bias for lower variance.

One approach described by Goodfellow et al. (2016, pp. 239–242) is to introduce parameter norm penalties which directly modify the training objective. A regularized estimator can be defined as

$$\hat{\theta} = \arg\min_{\theta} \frac{1}{n} \sum_{i=1}^{n} \ell(y_i, f_\theta(x_i)) + \alpha\Omega(\theta), \tag{2.9}$$

where $\Omega(\theta)$ penalizes selected parameter configurations and $\alpha \geq 0$ determines the strength of the penalty. A common choice is the L_2 penalty on the network weights

$$\Omega(\theta) = \frac{1}{2}\|\theta\|_2^2. \tag{2.10}$$

In neural network training, this is commonly referred to as weight decay and encourages parameter configurations with smaller weight magnitudes (Prince, 2023, pp. 138–140).

Goodfellow et al. (2016, p. 272) provide an important connection between stochastic optimization and regularization as they point out that even the choice of batch size $|B_t|$ may offer a regularizing effect as smaller batches can generalize better. Compared with full-batch gradient descent, SGD computes parameter updates from gradients that are generally noisier. This noise can have a regularizing effect, although smaller batches may require smaller learning rates. Prince (2023, pp. 143–144) provide an interpretation of this: the intuition is that SGD favors regions where different batches recommend parameter updates in similar directions, rather than regions where good average performance is driven mainly by a small number of batches. Such solutions fit the individual subsets of the training sample more uniformly and may therefore generalize better than solutions with the same average training loss but greater variation across batches. This phenomenon is known as implicit regularization. Contrary to parameter norm penalties (explicit regularization), the regularizing effect is induced not by changing the objective but through natural training dynamics.

Beyond the implicit regularization induced by optimization, regularization can also be introduced explicitly without adding a penalty to the objective. Dropout is a powerful regularization tool that randomly clamps a subset of hidden units to zero during each training iteration t. This has the effect of reducing dependence on particular hidden units and encourages weights to have less overall magnitude (Prince, 2023, pp. 147–148).

Each random dropout mask at iteration t creates a different subnetwork of the original neural network. Thus, each training iteration presents a mini-batch to a randomly sampled subnetwork. Goodfellow et al. (2016, pp. 251–257) show that dropout can thus be interpreted as a form of bagging because training effectively covers a large ensemble of subnetworks, although these subnetworks share parameters instead of being trained independently. At evaluation, all units are activated and their outgoing weights are multiplied by the retention probability $1 - q$, where q is the deactivation probability, thereby approximating the ensemble prediction in a single forward pass.

2.2 Smoother representations

Following Knaus (2024), a smoother is defined as a prediction rule whose fitted values can be represented as a weighted combination of observed outcomes. In this chapter, $y \in \mathbb{R}^N$ denotes the vector of training outcomes and $S \in \mathbb{R}^{N \times N}$ denotes the corresponding smoother matrix, so that fitted training predictions take the form:

$$\hat{y} = Sy. \tag{2.11}$$

Importantly, the term smoother does not refer to a specific estimation procedure. Rather, it refers to any prediction rule for which the fitted values can be represented as $\hat{y} = Sy$. Hence, the crucial requirement is not how the predictions are computed, but whether there exists a smoother matrix S such that the predicted outcomes can be written as a weighted transformation of the observed outcome vector. This definition also allows for adaptive smoothers, where the weights in S may themselves depend on the data used to construct the prediction rule (Knaus, 2024). For the purposes of this work, this means that the question is not whether a neural network is estimated by a classical smoothing algorithm, but whether its predictions can be represented exactly or approximately as weighted sample outcomes.

3 Methodology

This chapter develops the methodological framework used to represent neural network predictions as weighted outcomes. Building on the smoother concept introduced in the previous chapter, it considers two approaches. The first constructs an approximate smoother representation from the learned feature space of the trained network's final hidden layer (Coulombe et al., 2024). The second uses the telescoping representation of neural network training to express the prediction path as a sequence of linearized functional updates (Jeffares et al., 2024). The notation in this work is adapted from the original authors but may differ in form.

3.1 Neural feature kernel ridge regression

The first approach to obtaining a smoother representation of neural networks follows the approach of Coulombe et al. (2024). Recall that the output of a regression neural network can be expressed as a linear readout of the last hidden layer $\hat{y} = \begin{bmatrix} 1 & Z_{L-1} \end{bmatrix} \hat{\theta}_L$. This representation is important because it separates the neural network into two components: the hidden layers construct a nonlinear transformation of the original input, while the output layer is responsible for mapping this transformed representation linearly to the prediction $\hat{y}$. Neural networks are therefore not only prediction functions but also learners of internal feature representations. As pointed out by Goodfellow et al. (2016, p. 165), this perspective is closely related to kernel methods, which also map inputs into a specific feature space before applying a prediction rule. Compared to classical kernel methods, however, the feature transformation is not fixed in advance. Instead, the neural network learns its representations directly from the data through parameter optimization.

Smoother construction.

Coulombe et al. (2024) utilize this perspective by treating the linear readout of the output layer as a kernel ridge regression in neural network feature space. Let $\Phi(X) \in \mathbb{R}^{N \times p_l}$

denote the frozen feature matrix of the penultimate layer of the trained neural network, and let $\Phi(x_i) \in \mathbb{R}^{p_l}$ denote the corresponding feature vector of observation x_i such that

$$Z_{L-1} = \Phi(X) \qquad \text{and} \qquad Z_{i,L-1} = \Phi(x_i). \tag{3.1}$$

The representations are described as frozen because the network parameters have already been estimated and are held fixed during a forward pass. Thus, the corresponding neural network prediction can be written as

$$\hat{y}^{NN} = \Phi(X)\hat{\theta}_L. \tag{3.2}$$

As elaborated above, the linear form of the output layer suggests that $\Phi(X)$ can be treated as regressors in an auxiliary ridge regression. Following Coulombe et al. (2024), the neural network predictions for the full training sample are approximated in least-squares form as

$$\begin{aligned}
\hat{y}^{NN} &\approx \Phi(X)\left(\Phi(X)^\top\Phi(X) + \lambda I_{p_l}\right)^{-1}\Phi(X)^\top y && (3.3)\\
&= \Phi(X)\Phi(X)^\top\left(\Phi(X)\Phi(X)^\top + \lambda I_N\right)^{-1} y && (3.4)\\
&= K\left(K + \lambda I_N\right)^{-1} y. && (3.5)
\end{aligned}$$

Here, $\lambda > 0$ denotes the ridge penalty, while $I_{p_l} \in \mathbb{R}^{p_l \times p_l}$ and $I_N \in \mathbb{R}^{N\times N}$ denote the corresponding identity matrices. Furthermore, the neural feature kernel is defined as

$$K = \Phi(X)\Phi(X)^\top, \tag{3.6}$$

and finally, the predictions can be represented in smoother form by constructing the smoother matrix $S_\lambda \in \mathbb{R}^{N\times N}$ as

$$S_\lambda = K\left(K + \lambda I_N\right)^{-1}. \tag{3.7}$$

Thus, the predictions are an approximate linear transformation of the observed training outcomes y, with weights determined by similarities in the learned neural feature space $\Phi(X)$. Because its construction relies only on the linearity of the final readout, it applies irrespective of the architecture used to generate $\Phi(X)$, including dense, recurrent, convolutional, and attention-based networks, making it highly flexible.

Approximation.

The notation $\approx$ in (3.3) emphasizes that the least-squares ridge representation need not reproduce the original neural network predictions exactly. Coulombe et al. (2024) attribute this discrepancy to two related sources.

First, the output-layer weights $\hat{\theta}_L$ are learned jointly with the remaining network parameters $\hat{\theta}_{1:L-1}$ and are therefore not guaranteed to equal the full-sample least-squares solution conditional on $\Phi(X)$. For $\lambda = 0$, the representation in (3.3) would be exact if the gradient of the full-sample loss with respect to θ_L were zero. Under squared-error loss, this requirement yields the OLS normal equation

$$\Phi(X)^\top \left(y - \Phi(X)\hat{\theta}_L \right) = 0, \tag{3.8}$$

which states that the residual vector is orthogonal to the learned features (Hastie, 2024). Training the unregularized network to a global minimum is therefore sufficient, but not necessary, for this equality. However, common neural network training practices, including early stopping and stochastic mini-batch optimization, do not guarantee that the full-sample gradient with respect to θ_L is zero when training terminates. Consequently, the OLS first-order condition may remain unsatisfied, causing the neural network output to differ from the least-squares solution (Coulombe et al., 2024).

Second, the introduced ridge penalty modifies the least-squares solution in order to stabilize the inversion of the potentially ill-conditioned matrix $\Phi(X)\Phi(X)^\top + \lambda I_N$. This auxiliary ridge objective is not generally part of standard neural network training, which does not generally involve regressing y on $\Phi(X)$ in a separate output-layer step. Coulombe et al. (2024) use it primarily as a tool to make the closed-form solution numerically viable. Because the learned features are often highly collinear, applying λ is therefore an essential step.

However, Coulombe et al. (2024) argue that a small positive ridge penalty may improve the approximation. Beyond stabilizing the matrix inversion, it can capture part of the residual neural network regularization that moves $\hat{\theta}_L$ away from its unregularized least-squares solution. This intuition is supported by Goodfellow et al. (2016, pp. 243–245), who show that regularization through early stopping limits the length of the optimization trajectory and therefore how far the parameters can move from their initial values. For a linear model with a quadratic objective, restricting this trajectory produces a shrinkage

effect similar to an L_2 penalty. Consequently, introducing λ may reproduce the original neural network output more accurately than the unregularized solution obtained with $\lambda = 0$.

Post-processing modification.

Whereas Coulombe et al. (2024) select λ to make the auxiliary ridge predictions reproduce $\hat{y}^{NN}$ as closely as possible, this work selects λ according to validation performance and defines the resulting ridge predictor as a new estimator, $\hat{y}^{\text{ridge}}$. Conditional on the frozen feature matrix $\Phi(X)$, its readout coefficients are re-estimated by solving

$$\hat{\beta}_\lambda = \arg\min_{\beta \in \mathbb{R}^{p_l}} \left\{ \|y - \Phi(X)\beta\|_2^2 + \lambda \|\beta\|_2^2 \right\}. \tag{3.9}$$

Because $\hat{y}^{\text{ridge}}$ is defined by this ridge solution, its primal, kernel, and smoother representations hold exactly conditional on $\Phi(X)$ and the selected λ:

$$\begin{aligned}
\hat{y}^{\text{ridge}} &= \Phi(X)\hat{\beta}_\lambda && (3.10)\\
&= \Phi(X)\left(\Phi(X)^\top \Phi(X) + \lambda I_{p_l}\right)^{-1} \Phi(X)^\top y && (3.11)\\
&= K\left(K + \lambda I_N\right)^{-1} y && (3.12)\\
&= S_\lambda y. && (3.13)
\end{aligned}$$

This exact identity applies to the post-processed estimator and does not imply equality with $\hat{y}^{NN}$. For a separate evaluation sample X^*, with feature kernel $K^* = \Phi(X^*)\Phi(X)^\top$, the corresponding representation becomes

$$\hat{y}^{*,\text{ridge}} = K^*\left(K + \lambda I_N\right)^{-1} y. \tag{3.14}$$

Coulombe et al. (2024) identify this explicit post-processing strategy as a possible extension but leave its predictive performance relative to the original neural network as an open question. The trade-off introduced by this modification can be understood geometrically (Hastie, 2024): If $\Phi(X) = UDV^\top$, then the smoother matrix can be decomposed as

$$S_\lambda = U \operatorname{diag}\left(\frac{d_1^2}{d_1^2 + \lambda}, \ldots, \frac{d_r^2}{d_r^2 + \lambda}\right) U^\top. \tag{3.15}$$

Here, directions associated with large singular values are retained almost fully, whereas weakly represented directions with small singular values are shrunk more strongly. A moderate value of λ can therefore improve conditioning and reduce variance, but an excessively large value introduces substantial bias and may suppress genuinely predictive components (Hastie, 2024).

3.2 Telescoping representation of neural networks

The second approach considered in this work follows the telescoping representation of neural networks proposed by Jeffares et al. (2024). In contrast to the neural-feature ridge regression approach, this method does not impose a smoother structure directly on a learned feature representation. Instead, it approximates the original neural network predictor by decomposing the training trajectory into a sequence of functional updates and replacing each update by a first-order linearization around the parameters from the previous step. In this work, the telescoping representation is derived for a fully connected MLP trained with standard SGD under squared-error loss. To make the training dynamics explicit, this section writes neural network predictions as $f_{\theta_t}(x)$, where θ_t denotes the parameters after training step t. Evaluated on the full training sample, $f_{\theta_T}(X)$ corresponds to the fitted prediction vector previously denoted by $\hat{y}^{NN}$.

Telescoping predictor construction.

Consider $X \in \mathbb{R}^{N\times p}$, the full training input matrix, and $y \in \mathbb{R}^N$, the corresponding vector of observed targets. At training step $t \in \{1,\dots,T\}$, let the sampled mini-batch be defined as $B_t \subseteq \{1,\dots,N\}$, with batch size $|B_t| = b_t$. The corresponding input matrix and outcome vector are denoted by $X_t \in \mathbb{R}^{b_t\times p}$ and $y_t \in \mathbb{R}^{b_t}$. Furthermore, let $f_{\theta_t}(x_i)$ denote the network prediction for observation i after training update t.

Define $J_t(X)$ as the full-sample Jacobian evaluated immediately *before* update t, where p_θ is the number of trainable network parameters. Thus, each row contains the gradient of one network prediction with respect to all network parameters evaluated at step $t-1$:

$$J_t(X) = \begin{bmatrix} \nabla_\theta f_{\theta_{t-1}}(x_1)^\top \\ \vdots \\ \nabla_\theta f_{\theta_{t-1}}(x_N)^\top \end{bmatrix} \in \mathbb{R}^{N\times p_\theta}. \tag{3.16}$$

Lastly, define the batch-selection matrix $E_t \in \{0,1\}^{b_t\times N}$ such that the mini-batch Jacobian $J_t(X_t)$ can be written as $J_t(X_t) = E_t J_t(X)$. The matrix E_t selects the rows corresponding to the observations contained in B_t for any matrix or vector whose first dimension is N.

To establish how a parameter update translates into a change in the network prediction, first consider the underlying optimization step. At training step t, SGD updates

the parameter vector using the gradient of the current mini-batch loss evaluated at the pre-update parameters:

$$\theta_t = \theta_{t-1} - \gamma_t \nabla_\theta \mathcal{L}_{B_t}(\theta_{t-1}). \tag{3.17}$$

Define the corresponding parameter change as $\Delta\theta_t = \theta_t - \theta_{t-1}$. By the chain rule, the parameter update can then be written equivalently as

$$\begin{aligned}
\Delta\theta_t &= -\gamma_t \nabla_\theta \mathcal{L}_{B_t}(\theta_{t-1}) && (3.18)\\
&= -\frac{\gamma_t}{b_t} \sum_{i \in B_t} \nabla_\theta f_{\theta_{t-1}}(x_i) g_{it}^\ell && (3.19)\\
&= -\frac{\gamma_t}{b_t} J_t(X_t)^\top g_t^\ell, && (3.20)
\end{aligned}$$

where $g_{it}^\ell = \frac{\partial \ell\left(f_{\theta_{t-1}}(x_i), y_i\right)}{\partial f_{\theta_{t-1}}(x_i)}$ defines the derivative of the observation-level loss with respect to its prediction. For the considered squared-error loss, the derivative becomes $g_{it}^\ell = 2\left(f_{\theta_{t-1}}(x_i) - y_i\right)$. The vector $g_t^\ell \in \mathbb{R}^{b_t}$ collects these derivatives for observations in the current mini-batch. With the parameter update defined, the corresponding change in the prediction $\Delta f_t(x_i)$ at training observation x_i can be expressed as

$$\Delta f_t(x_i) = f_{\theta_t}(x_i) - f_{\theta_{t-1}}(x_i). \tag{3.21}$$

Jeffares et al. (2024) use a first-order Taylor expansion to linearize the prediction update around θ_{t-1}, which allows for the following approximation:

$$\begin{aligned}
\Delta f_t(x_i) &= \nabla_\theta f_{\theta_{t-1}}(x_i)^\top \Delta\theta_t + O\left(\|\Delta\theta_t\|^2\right) && (3.22)\\
&\approx \nabla_\theta f_{\theta_{t-1}}(x_i)^\top \Delta\theta_t && (3.23)\\
&= \Delta\tilde{f}_t(x_i). && (3.24)
\end{aligned}$$

While $\Delta f_t(x_i)$ is the actual change in the network prediction after the parameter update, $\Delta\tilde{f}_t(x_i)$ is used to denote its approximation. Intuitively, the Taylor expansion treats the network as locally linear in its parameters over the single update from θ_{t-1} to θ_t. The term $O(\|\Delta\theta_t\|^2)$ captures higher-order curvature effects and is discarded. Summing these approximated functional updates over the full training path yields the telescoping predictor $\tilde{f}_{\theta_T}(x_i)$ at final training step T:

$$\tilde{f}_{\theta_T}(x_i) = f_{\theta_0}(x_i) + \sum_{t=1}^{T} \nabla_\theta f_{\theta_{t-1}}(x_i)^\top \Delta\theta_t. \tag{3.25}$$

Here, $f_{\theta_0}(x_i)$ is the network's output evaluated at the initialization parameters θ_0. Evaluating the telescoping predictor over the complete training sample X and substituting the parameter update $\Delta\theta_t$ from (3.20) gives

$$\tilde{f}_{\theta_T}(X) = f_{\theta_0}(X) + \sum_{t=1}^{T} J_t(X)\Delta\theta_t \tag{3.26}$$

$$= f_{\theta_0}(X) - \sum_{t=1}^{T} \frac{\gamma_t}{b_t} J_t(X) J_t(X_t)^\top g_t^\ell \tag{3.27}$$

$$= f_{\theta_0}(X) - \sum_{t=1}^{T} \frac{\gamma_t}{b_t} \Theta_t g_t^\ell, \tag{3.28}$$

where the product

$$\Theta_t = J_t(X) J_t(X_t)^\top \in \mathbb{R}^{N \times b_t} \tag{3.29}$$

defines the empirical neural tangent kernel (eNTK) matrix between the full training sample and the mini-batch input X_t. Its entries measure the pairwise alignment between the parameter gradients of the full-sample predictions and those of the mini-batch predictions. It therefore describes how much a parameter update generated at time t and evaluated at the pre-update parameters θ_{t-1} affects every training prediction. Because the Jacobians evolve during training, the kernel provides a local similarity measure in parameter-gradient space that may change along the training trajectory (Jeffares et al., 2024).

Smoother construction.

Jeffares et al. (2024) show that the general telescoping predictor can be translated into the affine smoother representation $\tilde{f}_{\theta_t}(X) = S_t y + c^0_{\theta_t}(X)$, where $c^0_{\theta_t}(X)$ collects the contribution associated with the initial network prediction $f_{\theta_0}(X)$. In this work, however, the output-layer weights and bias are initialized at zero to omit the offset, rendering $f_{\theta_0}(X) = 0$. Under the vanilla SGD dynamics considered, the offset therefore remains zero throughout training and thus $c^0_{\theta_t}(X) = 0$ for all t. The initialization-dependent component can hence be ignored, yielding the following representation at iteration $t-1$:

$$\tilde{f}_{\theta_{t-1}}(X) = S_{t-1} y, \qquad S_0 = 0. \tag{3.30}$$

To derive the recursive smoother representation, Jeffares et al. (2024) explicitly assume that the telescoping approximation holds exactly at each training step, so that $\tilde{f}_{\theta_{t-1}}(X_t)$ can be substituted for $f_{\theta_{t-1}}(X_t)$ in the loss gradient.

Because E_t selects the observations belonging to the current mini-batch, the batch outcomes and predictions can be expressed as $y_t = E_t y$ and $\tilde{f}_{\theta_{t-1}}(X_t) = E_t S_{t-1} y$, respectively. The batch residual can therefore be expressed as

$$y_t - \tilde{f}_{\theta_{t-1}}(X_t) = (E_t - E_t S_{t-1})\, y. \tag{3.31}$$

Substituting this residual into the squared-loss gradient yields $g_t^{\ell} = -2\,(E_t - E_t S_{t-1})\, y$ which, when substituted into the functional update, gives

$$\Delta \tilde{f}_t(X) = \frac{2\gamma_t}{b_t} \Theta_t\, (E_t - E_t S_{t-1})\, y, \tag{3.32}$$

and therefore

$$\tilde{f}_{\theta_t}(X) = \tilde{f}_{\theta_{t-1}}(X) + \Delta \tilde{f}_t(X) \tag{3.33}$$

$$= \underbrace{\left(S_{t-1} + \frac{2\gamma_t}{b_t} \Theta_t\, (E_t - E_t S_{t-1}) \right)}_{S_t} y. \tag{3.34}$$

Consequently, the smoother matrix is updated recursively like the network prediction along the training trajectory according to

$$S_t = S_{t-1} + \frac{2\gamma_t}{b_t} \Theta_t\, (E_t - E_t S_{t-1})\,. \tag{3.35}$$

After the final training step T, the telescoping predictions can therefore be written as

$$\tilde{f}_{\theta_T}(X) = S_T y. \tag{3.36}$$

For a test input matrix X^*, the same construction can be applied by tracking how the parameter updates generated during training affect the test predictions. Let $J_t(X^*)$ denote the test sample Jacobian evaluated at the pre-update parameters θ_{t-1}, and define the corresponding test-batch tangent kernel as $\Theta_t^* = J_t(X^*) J_t(X_t)^\top$. The test smoother is then updated with the same training residuals as

$$S_t^* = S_{t-1}^* + \frac{2\gamma_t}{b_t} \Theta_t^*\, (E_t - E_t S_{t-1})\,. \tag{3.37}$$

It is important to note that this procedure does not leak test information into training. The parameter updates are still computed exclusively with the training batches and their residuals. The test set X^* is only evaluated passively through $J_t(X^*)$ in order to track how the training-induced parameter updates would change the corresponding test predictions.

Approximation.

The approximation is required to translate the trained neural network into a smoother representation. Since the network is nonlinear in its parameters, the telescoping approach replaces each nonlinear functional update with a first-order Taylor approximation. This allows the training trajectory to be accumulated update by update and expressed through a linear combination of gradients and parameter updates (Jeffares et al., 2024). The approximation quality is primarily driven by the size of the individual parameter updates $\Delta\theta_t$. Because the Taylor remainder is of order $O(\|\Delta\theta_t\|^2)$, the approximation becomes more accurate when each update is small. Consequently, Jeffares et al. (2024) emphasize that the learning rate γ_t is of central importance in governing the approximation accuracy, since it directly affects the magnitude of the parameter update. Assuming that the telescoping approximation holds exactly then makes the smoother recursion S_t viable, because the current network predictions inside the loss gradient can be replaced by $\tilde{f}_{\theta_{t-1}}$, yielding predictions of the form $\tilde{f}_{\theta_T}(X) = S_T y$.

The gradient interactions are summarized by $\Theta_t = J_t(X)J_t(X_t)^\top$, which is an empirical neural tangent kernel because it is computed from the realized Jacobians of the finite network at training step t. This kernel is therefore time-dependent: as the parameters change during training, the Jacobians and hence Θ_t may change as well. This kernel interpretation of the linearization is related to lazy-learning and theoretical NTK perspectives, but it is less restrictive. In the lazy-learning regime, the parameter gradients remain approximately constant during training:

$$\nabla_\theta f_{\theta_t}(x) \approx \nabla_\theta f_{\theta_0}(x), \tag{3.38}$$

which implies that the tangent kernel also remains approximately constant, $\Theta_t \approx \Theta_0$ (Golikov et al., 2022). The network can then be analyzed through its linearization around initialization. Jacot et al. (2020) show that for infinite-width networks, the NTK converges to a deterministic limiting kernel that remains constant during training. Under this infinite-width constant-kernel limit, the evolution of the network function follows kernel gradient descent in function space. For finite networks, this interpretation depends on the empirical NTK remaining sufficiently close to its initial value under suitable parameterization (Golikov et al., 2022).

As Jeffares et al. (2024) point out, the telescoping approach relaxes this lazy-learning requirement by re-linearizing after each training update rather than only once around

initialization. Thus, it does not require the empirical kernel Θ_t to remain globally constant over the full training path. Instead, it only requires each local update to be small enough for the first-order approximation around θ_{t-1} to be accurate. Network width is therefore relevant mainly as theoretical motivation: wider networks may move the eNTK closer to the constant-kernel behavior predicted by NTK theory, but in the telescoping representation the practical driver of approximation quality remains the update size, and hence the learning rate.

Computational limitations.

This construction should be interpreted as an approximation rather than as a separate training procedure. Jeffares et al. (2024) present the method as an empirical analysis tool for studying neural network learning rather than as an alternative to standard neural network training. Its main practical limitation is its computational cost. Standard neural network training requires only the gradient of the aggregated mini-batch loss at each update. In contrast, the telescoping construction requires evaluating the gradient of the network output with respect to all model parameters separately for every observation. These Jacobians must be recomputed after each training step because they change with the network parameters. Constructing S_T additionally requires computing tangent-kernel matrices and recursively updating an $N \times N$ smoother matrix. Moreover, including test observations introduces further Jacobian and cross-kernel computations. Thus, the method adds substantial overhead compared to ordinary neural network training and may become prohibitive for very large networks, datasets or long training trajectories.

4 Empirical evaluation

This chapter empirically evaluates the two smoother representations introduced in the previous chapter. The objective is to assess how well each approach performs in its respective setting rather than to find out which of the two performs better. Because the approaches are evaluated under different experimental conditions, such a direct comparison would not be meaningful. The evaluation focuses on two aspects. First, it assesses the approximation agreement, i.e., the extent to which the smoother representation reproduces the original network's predictions. Second, it examines the predictive accuracy of the underlying neural networks and their smoother-based counterparts.

In order to assess robustness, each method is evaluated across multiple runs that vary in hyperparameters, network-initialization seeds, and train-validation-test split seeds. Each run produces a new set of test predictions for the original neural network and its corresponding ridge or telescoping approximation. Error metrics are calculated separately for every run and then summarized across runs. All experiments were executed on uniClusterbw. The supplementary code is accessible at `https://github.com/aaronsennhenn/neural_networks_as_approximate_smoothers`.

4.1 Experimental setup

Data.

The analysis uses two standard OpenML regression datasets proposed by Grinsztajn et al. (2022), namely the Concrete Compressive Strength dataset and the California Housing dataset. Each is divided into separate training, validation, and test sets using four different split seeds. The training set is used to fit the neural network model. The validation set is used both for validation during neural-network training and for selecting the ridge penalty. The final performance is reported on the test set. The California experiments use a subset of $N = 5{,}000$ for ridge and $N = 1{,}000$ for telescoping.

Preprocessing.

The preprocessing procedure aims to preserve the outcome-weight interpretation of the smoother representation. In particular, only features are standardized, while targets remain on their original scale. This is necessary because the smoother $\hat{y} = Sy$ is defined with respect to the original target vector. Standardizing targets and rescaling after final prediction would introduce an additional offset and therefore interfere with this representation.

At the same time, unscaled targets may lead to unstable optimization, especially for datasets with large target values. For the neural-feature ridge smoother, this is avoided by using the Adam optimizer, which internally rescales gradients and thus makes training stable (Goodfellow et al., 2016, p. 306).

For simplicity, the telescoping smoother is trained with standard SGD. In order to improve numerical stability and avoid exploding gradients, the training loss is re-scaled by a factor proportional to the inverse of the training target standard deviation σ_y. For SGD, scaling the loss is practically equivalent to scaling the parameter update step $\Delta\theta_t$ by that factor:

$$\Delta\theta_t = -\frac{\gamma_t}{\sigma_y}\nabla_\theta \mathcal{L}_{B_t}(\theta_{t-1}). \tag{4.1}$$

Additionally, for the California Housing dataset the target *Median house value* is divided by a factor of 10,000. Thus, metrics are reported in units of \$10,000.

Hyperparameters.

For the ridge smoother, the neural network is trained over a predefined hyperparameter grid. The ridge penalty is selected from a fixed grid of candidate values using the validation data. The network parameters are initialized using three different random seeds. Together with the four data-split seeds and the different combinations of batch size, hidden-layer width, and learning rate, this results in a total of 144 experimental configurations.

The telescoping smoother setup is more restricted due to the computational requirements. The telescoping smoother is evaluated over only 12 runs varying in data splits and initialization seeds while using a single fixed hyperparameter set. This set is motivated by the role of small parameter updates in the telescoping approximation and follows the empirical considerations for hyperparameters discussed in Jeffares et al. (2024).

4.2 Evaluation

Evaluation is based on mean absolute error (MAE), root mean squared error (RMSE), mean absolute percentage error (MAPE), and Q95, which denotes the within-run 95th percentile of the absolute test errors. MAPE is reported as a proportion, i.e., 0.230 corresponds to 23.0%. For the approximation agreement analysis, MAPE is calculated relative to the neural network predictions rather than the observed targets.

Approximation agreement.

Approximation agreement compares each smoother-based predictor with its corresponding neural network predictor. For telescoping, this measures the error introduced by the first-order Taylor expansion of the training trajectory. For neural-feature ridge, the smoother identity is exact conditional on the frozen features and selected penalty. Thus, disagreement with the original neural network instead measures the change induced by refitting and regularizing its final readout in post-processing.

Table 4.1 shows that the telescoping predictor has an average agreement MAPE of 1.09% on California and 0.49% on Concrete. The ridge post-processing changes the original network predictions by an average of 5.07% and 3.50%, respectively. The run-level boxplots in Figure 4.2 support this: Ridge disagreement is more widely dispersed across runs with occasional outliers, whereas telescoping approximation errors are concentrated closer to zero in most runs. Prediction-level scatter plots in Figure 4.1 show that at

Method	**Dataset**	**MAE**	**RMSE**	**MAPE**	**Q95**
Ridge	California	0.996 (0.448)	1.298 (0.553)	0.051 (0.021)	2.562 (1.081)
Ridge	Concrete	1.125 (0.709)	1.398 (0.867)	0.035 (0.022)	2.693 (1.719)
Tel.	California	0.217 (0.101)	0.294 (0.137)	0.011 (0.004)	0.525 (0.205)
Tel.	Concrete	0.184 (0.319)	0.228 (0.359)	0.005 (0.008)	0.417 (0.659)

Table 4.1: Approximation agreement across runs. Reported as mean (sd).

the observation level the ridge smoother has occasional individual outliers, but the bulk of predictions stay close to the original network's output. This pattern reflects the designs of the two constructions. Telescoping follows the network's training path and is intended to reproduce its predictions. Ridge instead replaces the trained readout with a

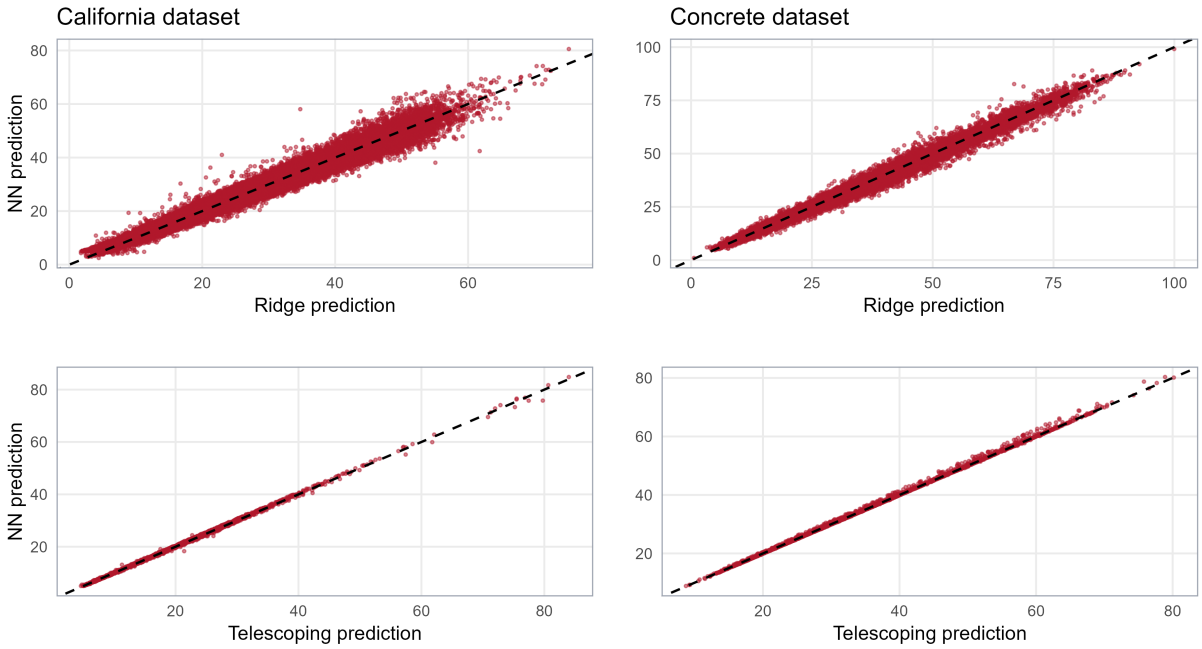

Figure 4.1: Prediction-level approximation agreement between smoother predictions and the corresponding neural-network predictions.

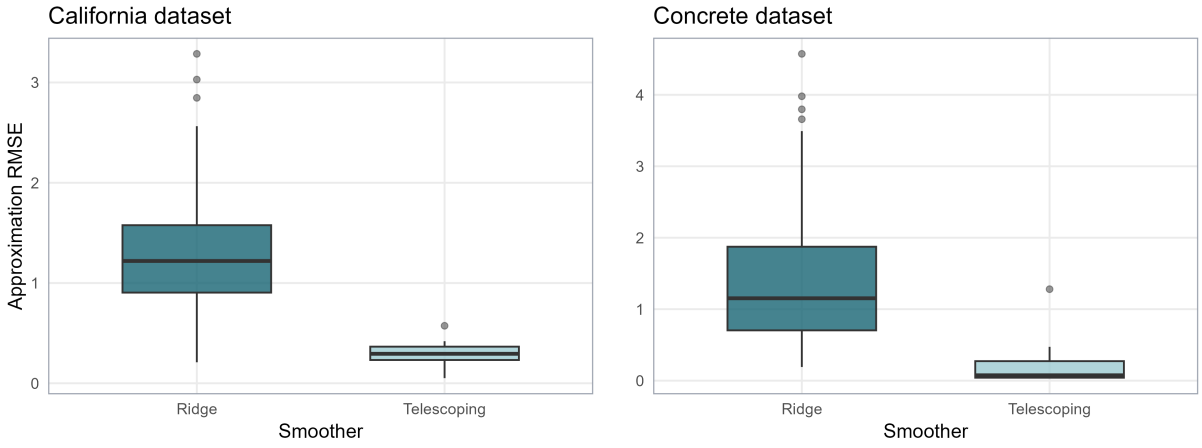

Figure 4.2: Run-level distribution of approximation RMSE.

validation-selected penalized readout. Its larger and more variable disagreement therefore reflects the deliberate construction of a new estimator rather than an inaccurate smoother representation.

Predictive performance.

Across the ridge runs, Table 4.2 shows that the ridge smoother reduced all four average error measures by a slight amount compared to the original neural networks. This small improvement is expected, considering the design of the method: The final readout is re-estimated, and its ridge penalty is selected based on predictive performance on a validation set, although improved performance is not guaranteed in every run.

For telescoping, none of the average performance differences is meaningful. The approximation therefore largely preserves the predictive performance of the corresponding neural networks. Figure 4.3 supports these findings: Ridge runs cluster slightly above the equality line, which indicates improved performance relative to the corresponding network. The telescoping observations stay closely around it.

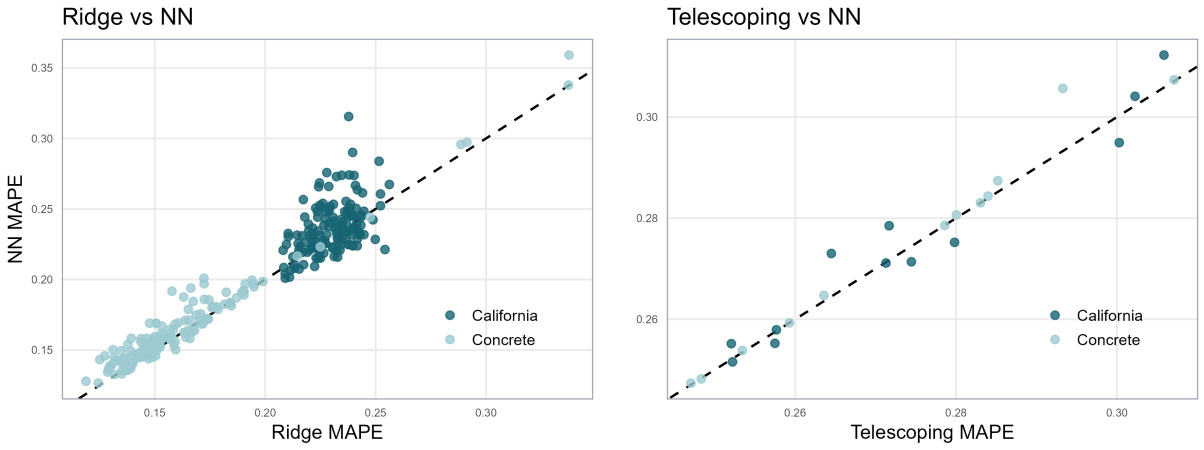

Figure 4.3: Run-level comparison of MAPE.

Overall, both constructions preserve the predictive performance of their corresponding neural networks under the evaluated settings. Ridge refitting slightly improves average test performance, while telescoping closely reproduces the original network predictions.

		MAE		RMSE		MAPE		Q95	
Method	**Dataset**	**SM**	**NN**	**SM**	**NN**	**SM**	**NN**	**SM**	**NN**
Ridge	California	3.995	4.112	5.782	5.904	0.230	0.236	11.818	12.124
		(0.161)	(0.203)	(0.264)	(0.289)	(0.011)	(0.019)	(0.535)	(0.710)
Ridge	Concrete	4.751	4.916	6.422	6.611	0.161	0.166	13.003	13.423
		(0.759)	(0.752)	(0.950)	(0.950)	(0.033)	(0.034)	(2.070)	(2.130)
Tel	California	5.003	5.005	7.005	6.995	0.274	0.275	13.353	13.371
		(0.281)	(0.281)	(0.341)	(0.340)	(0.019)	(0.020)	(0.755)	(0.781)
Tel	Concrete	7.181	7.194	9.223	9.251	0.274	0.275	17.905	18.048
		(0.312)	(0.334)	(0.448)	(0.500)	(0.019)	(0.020)	(1.821)	(2.050)

Table 4.2: Predictive performance across runs. For the original neural network (NN) and the corresponding smoother (SM). Reported as mean (sd).

5 Application: Outcome-Weights

This chapter examines whether the neural-feature ridge smoother can be used as an outcome nuisance learner in the Outcome-Weights framework of Knaus (2024) by replicating their 401(k) application. To this end, it utilizes the derived neural-feature ridge smoother. The telescoping smoother is not considered here because its requirements make this application computationally prohibitive. Because the ridge predictor is designed such that $\hat{y} = S_\lambda y$ holds exactly, it satisfies the smoother requirement of Knaus (2024) by construction. Consequently, the corresponding DML estimator admits an exact weighted-outcome representation $\widehat{\tau} = \omega^\top Y$. The purpose of the application is therefore not to re-establish this algebraic identity, but to examine whether the implied outcome weights provide useful diagnostics.

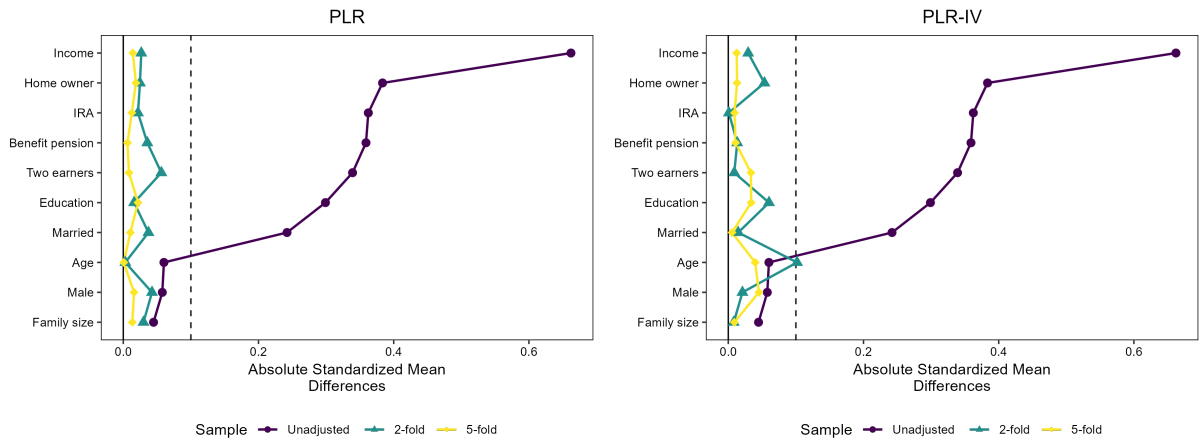

Figure 5.1: Absolute standardized mean differences for the 401(k) covariates.

Figure 5.1 examines the balancing properties of the implied outcome weights. The maximum absolute standardized mean difference decreases after applying the neural-network outcome weights, with all ten covariates falling below the threshold of 0.1. Hence, the neural-feature construction makes the outcome-weight diagnostics proposed by Knaus (2024) available for a neural-network-based nuisance estimator.

6 Conclusion

This thesis examined two approaches for approximately representing neural network predictions as smoothers. To this end, it derived and implemented a neural-feature ridge approach building on Coulombe et al. (2024) and a telescoping approach based on Jeffares et al. (2024). Overall, both approaches performed well in their intended settings and preserved predictive performance. Neural-feature ridge slightly improved average accuracy, although refitting the final readout introduced small deviations from the original network predictions. In contrast, telescoping reproduced these predictions closely but required substantially greater computational effort. Neural-feature ridge therefore provides a practical, exact smoother for a modified estimator, while telescoping offers a more faithful approximation of the original network. The application demonstrated how neural-feature ridge can be used as a nuisance estimator within the DML Outcome-Weights framework of Knaus (2024). This makes it possible to derive and examine outcome weights for a neural network-based nuisance learner. Future research should evaluate the scalability of both approaches to larger settings and investigate their statistical properties in causal estimation.

References

Chernozhukov, V., Chetverikov, D., Demirer, M., Duflo, E., Hansen, C., Newey, W., and Robins, J. (2018). Double/debiased machine learning for treatment and structural parameters. *The Econometrics Journal*, 21(1):C1–C68.

Coulombe, P. G., Goebel, M., and Klieber, K. (2024). Dual interpretation of machine learning forecasts. Retrieved from `https://arxiv.org/abs/2412.13076`.

Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2:303–314.

Golikov, E., Pokonechnyy, E., and Korviakov, V. (2022). Neural tangent kernel: A survey. Retrieved from `https://arxiv.org/abs/2208.13614`.

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. The MIT Press. Retrieved from `http://www.deeplearningbook.org`.

Grinsztajn, L., Oyallon, E., and Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on tabular data? Retrieved from `https://arxiv.org/abs/2207.08815`.

Hastie, T. (2024). Ridge regularization: an essential concept in data science. Retrieved from `https://arxiv.org/abs/2006.00371`.

Jacot, A., Gabriel, F., and Hongler, C. (2020). Neural tangent kernel: Convergence and generalization in neural networks. Retrieved from `https://arxiv.org/abs/1806.07572`.

Jeffares, A., Curth, A., and van der Schaar, M. (2024). Deep learning through a telescoping lens: A simple model provides empirical insights on grokking, gradient boosting & beyond. Retrieved from `https://arxiv.org/abs/2411.00247`.

Knaus, M. C. (2024). Treatment effect estimators as weighted outcomes. Retrieved from `https://arxiv.org/abs/2411.11559`.

Prince, S. J. (2023). *Understanding Deep Learning*. The MIT Press. Retrieved from `http://udlbook.com`.

Suh, N. and Cheng, G. (2025). A survey on statistical theory of deep learning: Approximation, training dynamics, and generative models. *Annual Review of Statistics and Its Application*, 12(Volume 12, 2025):177–207.